Oficina
Dossiê técnico da Fase 3

Arquitetura, componentes, sequências, modelo relacional, RFCs, ADRs e validação do ambiente de homologação.

Revisão de 31/10/2026 (BRT)

Este documento registra a solução observada e suas pendências. Não declara conformidade integral e não substitui o PDF final de submissão enquanto faltar o vídeo e a conclusão dos requisitos técnicos.

O vídeo será gravado pelo usuário após finalizar as pendências. Nenhum CPF, token, senha ou dado de cliente consta deste dossiê.

Fase 3 - documentação da Oficina

Pacote arquitetural e roteiro de homologação. Revisão: 30/10/2026 (BRT). O estado descrito é o ambiente acadêmico implantado. A matriz de validação diferencia implementação, evidência observada e pendências; documentação não substitui um teste operacional.

Leitura e apresentação

1. Arquitetura, componentes e fronteiras.
2. Sequências de autenticação e abertura de OS.
3. Modelo relacional, justificativa e evolução.
4. Matriz dos requisitos e evidências.
5. Roteiro do vídeo de até 15 minutos.
6. Folha de entrega e links.

Decisões

Documento	Tema
RFC-001	AWS, EKS e serviços gerenciados no Learner Lab
RFC-002	Escolha formal do PostgreSQL e consistência
RFC-003	CPF, JWT RS256, Gateway e RBAC
ADR-001	HTTP síncrono e fronteiras de acesso
ADR-002	HPA e diferença entre pods e nós
ADR-003	Stack compartilhada, dois dashboards e armazenamento efêmero

Os RFCs formalizam a justificativa da solução existente e registram os pontos em aberto. Os ADRs documentam decisões materializadas no código; não simulam aprovação histórica da equipe. Os ADRs existentes da aplicação continuam válidos nos respectivos escopos.

Fontes e propriedade

- Aplicação: Quarkus, domínio, REST, migrations e manifests da aplicação.

- Autenticação: funções Go e Terraform correspondente.

- Kubernetes: EKS, Gateway, VPC Link e observabilidade.

- Banco: RDS e controles de rede do banco.

Diagramas Mermaid estão versionados em `diagramas/*.mmd`; as imagens SVG/PNG permitem apresentação fora do GitHub. O modelo foi confrontado com entidades/migrations, sem consultar tabelas nem dados de clientes. Nenhuma credencial ou dado de cliente integra este pacote.

Dossiê para leitura offline

PDF técnico desta revisão. Os textos usam A4; os quatro diagramas usam páginas A3 para preservar a legibilidade. O PDF é um dossiê pré-entrega: o vídeo e as lacunas técnicas continuam pendentes.

Para reproduzir a partir da raiz do repositório, usar Python com Pillow/ReportLab e Graphviz: `python3 docs/fase3/gerar-diagramas.py` e `python3 docs/fase3/gerar-dossie.py`.

Arquivos separados

Os PDFs individuais ficam em `output/pdf/separados/`, gerados por `python3 docs/fase3/gerar-separados.py`. Incluem arquitetura, sequências, modelo de dados, cada RFC/ADR, matriz, roteiro simples e folha de entrega. Diagramas PNG/SVG e fontes Mermaid continuam em `docs/fase3/diagramas/`. O pacote ZIP reúne os PDFs e as fontes editáveis. O enunciado continua exigindo vídeo e PDF único de links para o Portal.

Arquitetura e componentes

Escopo e estado

A solução organiza atendimento, clientes, veículos, ordens de serviço, orçamentos, serviços e peças. O ambiente confirmado é homologação em AWS us-east-1 no Learner Lab. Os quatro repositórios possuem workflows e regras de PR em main. Os workflows de produção estão definidos; não foi encontrada execução de produção nesta revisão.

Caminhos de execução

Origem	Destino	Contrato e responsabilidade
Cliente externo	API Gateway HTTP	HTTPS; entrada pública, roteamento e integração com Authorizer
POST /v1/auth/customer	Lambda customer-auth	Valida CPF, consulta existência/status no PostgreSQL e emite JWT
Rotas protegidas	Lambda jwt-authorizer	Verifica JWT; REQUEST, payload 2.0, resposta simples e cache zero
Gateway	VPC Link -> NLB interno -> aplicação	HTTP privado até a API em 8080; a aplicação aplica RBAC e regras de negócio
Gateway /observability	VPC Link -> NLB interno -> Grafana	Login próprio do Grafana; a rota não usa o JWT da aplicação
API / Lambda customer-auth	RDS PostgreSQL 16	Banco privado, TLS; consultas de autenticação e persistência de domínio
Prometheus	Service de management da API	ClusterIP em 9000; métricas /q/metrics; sem exposição pública desse listener
Blackbox	Gateway /q/swagger-ui/	Sondagem pública periódica; mede disponibilidade dessa rota, não todas as regras de negócio
Alloy	Loki / Tempo	Logs dos pods e OTLP; serviços internos
Lambdas / Gateway	CloudWatch	Logs operacionais; tracing OTLP da Lambda é opcional e não foi ativado por esta documentação

As rotas OpenAPI/Swagger de homologação são exceções públicas explícitas. Management, Prometheus, Alertmanager, Loki, Tempo e receptor OTLP não possuem rota pública própria. Os NLBs de aplicação e Grafana são internos; o ELB público legado já estava ausente na revisão.

Responsabilidade de cada repositório

Repositório	Recursos e entrega
oficina-app	Java 21, Quarkus, API, domínio hexagonal, Dockerfile, imagem ECR, manifests/Kustomize, rollout, smoke e contrato upstream
oficina-lambda-auth	Go, dois binários Lambda, autenticação CPF, validação JWT, pacote ZIP, publicação de versão e contrato authorizer
oficina-infra-k8s	Terraform de EKS/ECR/Gateway/VPC Link; stack Helm; listener Grafana e reconciliação das rotas
oficina-infra-db	Terraform de RDS PostgreSQL e rede; state independente por ambiente

Entrega e convergência

PRs para main/homolog/production passam pelas verificações definidas por cada repositório. Push em homolog/production aciona os workflows de entrega correspondentes. A stack de observabilidade usa disparo manual serializado porque é compartilhada pelo cluster.

A aplicação publica `upstream.json`; a Lambda publica `authorizer.json`; observabilidade publica `grafana.json`. Os contratos ficam em S3 e contêm referências de integração e SHA, sem valores de secrets. A reconciliação preserva o outro ambiente e recusa planos fora dos recursos de roteamento autorizados. Terraform usa plano salvo e lock no backend S3. O bucket/região da publicação são transmitidos por outputs para o workflow reutilizável, evita-se resolver variáveis de outro escopo na chamada.

Segurança e limites reais

- O Gateway valida identidade; a API controla papel e acesso a cada recurso. CUSTOMER não recebe acesso operacional apenas por possuir JWT válido.

- CPF é um identificador de conhecimento e não um segundo fator. A solução atende ao fluxo acadêmico; operação comercial exige avaliar autenticação adicional, abuso e revogação.

- Segredos são fornecidos por GitHub Environments/Organization e montados/injetados nos workloads. State Terraform pode conter material sensível e não deve ser publicado.

- RDS é privado. O cliente reativo da API usa TLS com validação da cadeia relaxada no Lab; uma implantação comercial deve validar a CA e o hostname.

- ClusterIP não é isolamento entre pods. Não foi comprovado enforcement de NetworkPolicy nesta revisão.

- Homolog/production compartilham cluster e stack de monitoramento; namespaces não equivalem a isolamento físico.

- RDS Single-AZ, HPA mínimo de uma réplica e armazenamento efêmero de telemetria limitam disponibilidade/durabilidade. Não se afirma alta disponibilidade corporativa.

- O HPA escala pods. Não há comprovação de Cluster Autoscaler para criação automática de nós.

Base técnica conferida

- Aplicação: `f56897c0a4959d1ed3576d15ccc1920d9bd53e24` (homolog).

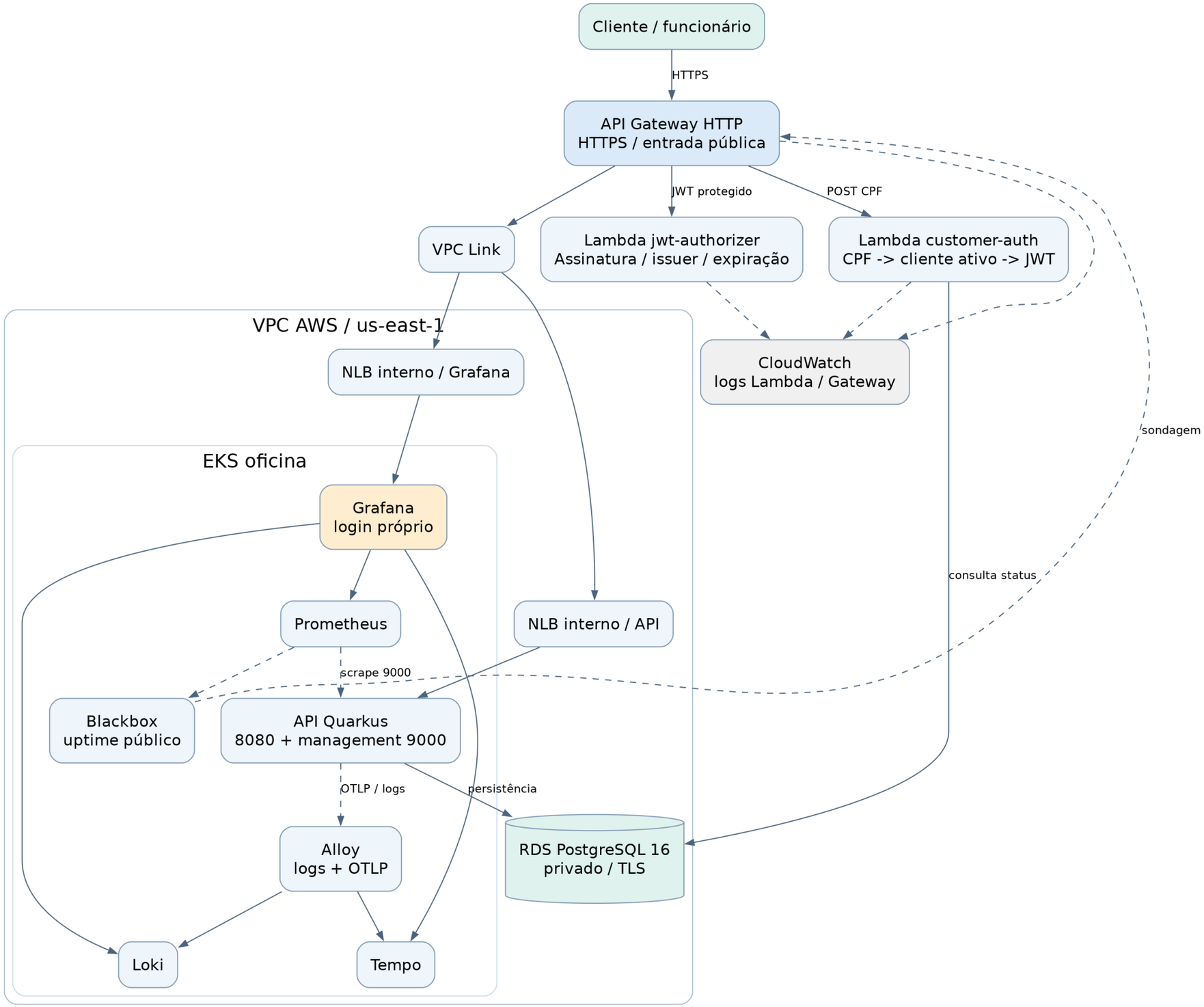
- Lambda: `8c0d4320eafeeb5b242020e76bfeee4be49f66e4` (homolog).

- Banco: `657489d3d6a932813c41835ec78d79b2bfac8838` (homolog).

- Kubernetes/observabilidade: PRs 26-29 e workflows registrados na matriz de validação.

Referências verificáveis: API, Gateway, Lambda, RDS.

Componentes da solução



Fonte editável e versão vetorial no diretório docs/fase3/diagramas. Diagrama de homologação; limites e caminhos alternativos no texto.

Sequências dos fluxos obrigatórios

CPF, emissão de token e consumo protegido

1. O cliente envia o CPF à rota pública POST /v1/auth/customer via TLS.

2. O Gateway integra essa rota com a Lambda customer-auth, sem exigir um token prévio.

3. A função valida o documento e consulta o cliente no PostgreSQL privado.

4. Ausência/inatividade impedem emissão. Com cliente ativo, emite JWT RS256 com issuer,

expiração, identidade e grupo CUSTOMER. Nenhum valor de CPF/JWT deve aparecer no vídeo.

5. Ao chamar GET /v1/me/work-orders, o cliente envia Authorization Bearer.

6. O Gateway pode rejeitar ausência de identidade com 401 sem invocar a função. Com

identidade presente, a Lambda Authorizer valida o token e responde permitir/negar.

7. A aplicação valida o contexto de segurança e o vínculo do recurso ao cliente.

O Authorizer não consulta o banco a cada requisição. Desativação de cliente não equivale a revogação instantânea de tokens já emitidos. O cache do Authorizer está em zero, mas isso não altera a validade criptográfica/temporal do JWT.

Abertura da ordem de serviço

Pré-condição: funcionário autenticado com papel ADMIN ou MECHANIC. O login de funcionário é POST /v1/auth/login na aplicação. O JWT CUSTOMER do fluxo anterior não autoriza este POST.

1. O funcionário envia POST /v1/work-orders com Bearer e referências de cliente/veículo.

2. O Gateway invoca o Authorizer; token recusado encerra o fluxo sem executar o caso de uso.

3. A aplicação aplica RBAC e valida o DTO. Identidade aceita pelo Gateway não substitui RBAC.

4. WorkOrderService.create executa com @WithTransaction. Consulta cliente/veículo e o

funcionário atribuído quando fornecido, e abre a OS em RECEIVED.

5. Quando existem serviços/peças iniciais, resolve catálogo, cria linhas/orçamento e

registra a evolução correspondente. Sem solicitação inicial, a OS não cria orçamento.

6. O adaptador de notificação prepara o acompanhamento via SMTP. Falhas de entrega são

tratadas pelo adaptador; não há Lambda de notificações implantada nesta arquitetura.

7. A transação é confirmada antes de a resposta HTTP 201 ser observada pelo cliente,

incluindo Location. Falhas transacionais não devem produzir uma OS parcial.

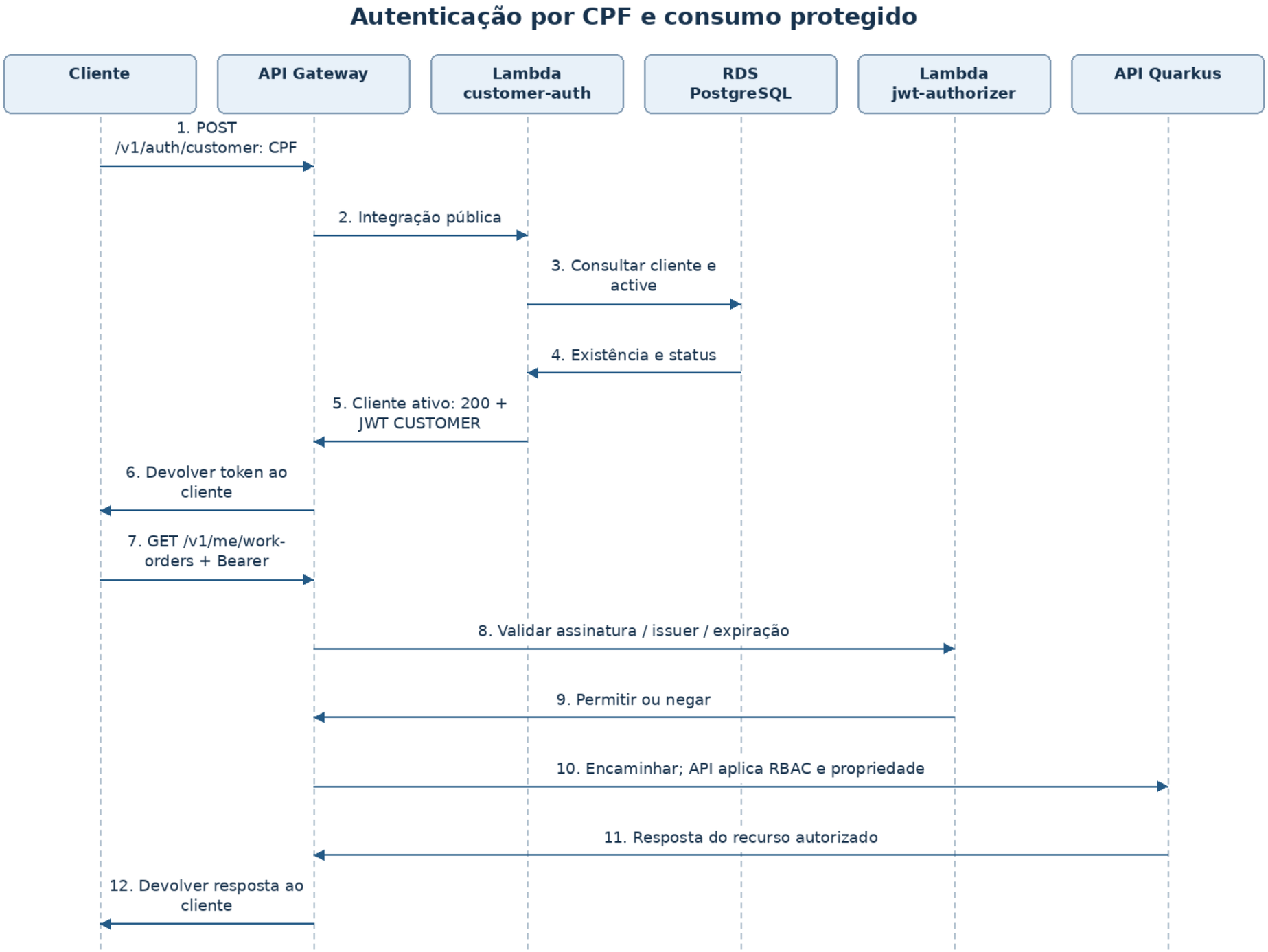
A chamada de notificação ocorre no fluxo reativo do caso de uso; este diagrama não promete padrão outbox, fila, entrega exatamente uma vez nem commit anterior ao envio SMTP.

Diagramas editáveis

Os arquivos `.mmd` correspondentes incluem os caminhos alternativos de erro e são renderizáveis no GitHub/Mermaid. As imagens exportadas destacam o caminho de sucesso e as recusas essenciais.

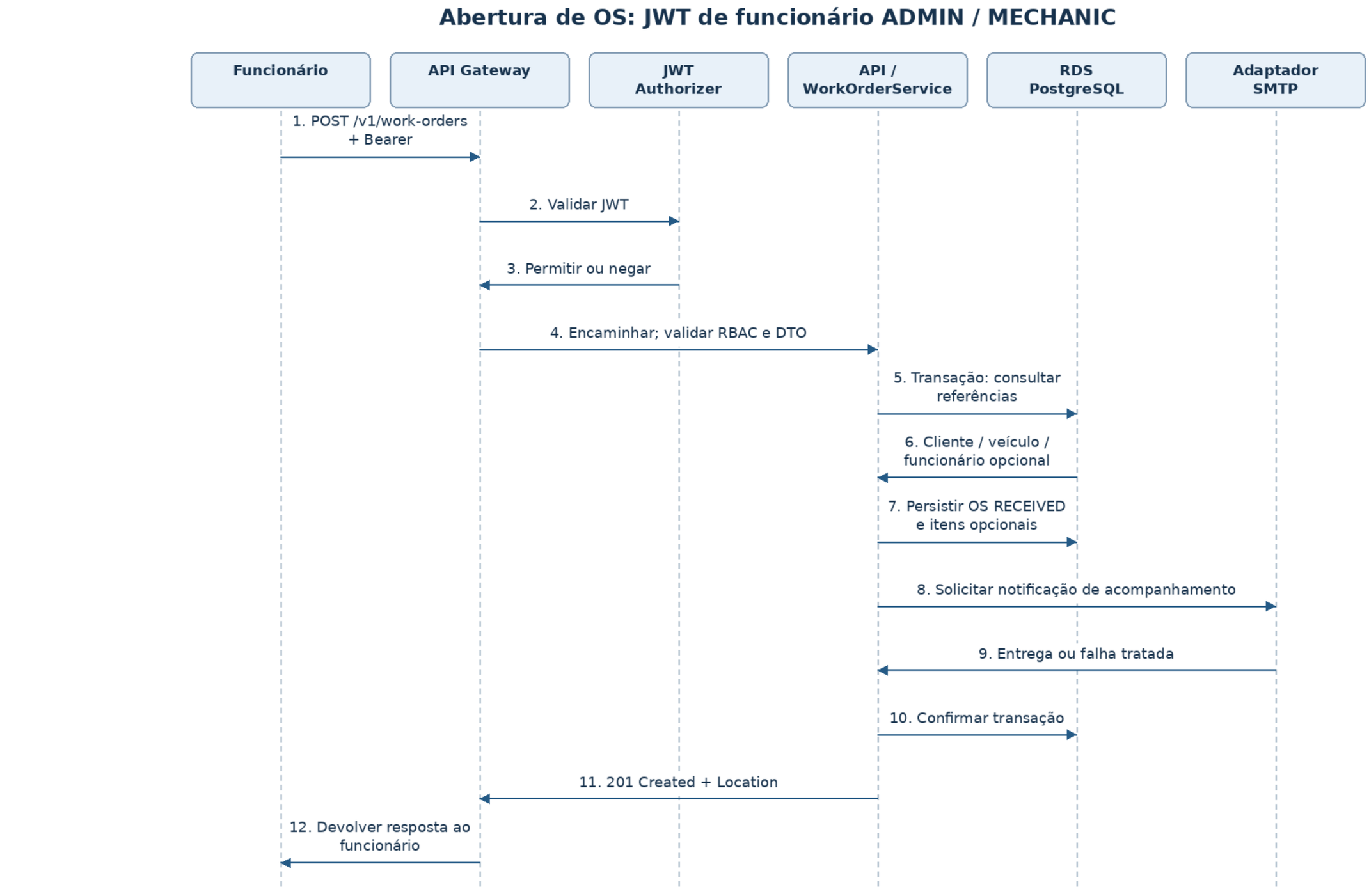
Fonte: WorkOrderController, WorkOrderService e handler CPF.

Autenticação CPF



Fonte editável e versão vetorial no diretório docs/fase3/diagramas. Diagrama de homologação; limites e caminhos alternativos no texto.

Abertura de OS



Pré-condição: login do funcionário. Falha transacional exige rollback; CUSTOMER não autoriza este POST.

Fonte editável e versão vetorial no diretório docs/fase3/diagramas. Diagrama de homologação; limites e caminhos alternativos no texto.

Banco de dados e modelo relacional

Justificativa formal do PostgreSQL gerenciado

A Oficina mantém relações fortes entre cliente, veículo, OS, orçamento, itens e estoque. A abertura de OS e a reserva/devolução de peças exigem transações e integridade entre entidades. PostgreSQL fornece transações ACID, chaves/constraints e consultas relacionais compatíveis com Hibernate Reactive/Panache, JDBC/Flyway e o driver pgx da Lambda. RDS delega operação do mecanismo e backups configurados ao serviço gerenciado.

Alternativas consideradas: MySQL gerenciado também atende relações/transações, mas exigiria adaptar tipos, drivers e migrations sem benefício medido para este projeto. Um banco NoSQL exigiria redesenhar as consultas e a consistência entre agregados. Não foi realizado benchmark comparativo; a escolha é de adequação funcional e continuidade da stack, não superioridade de desempenho.

Modelo implementado

Tabela	Identidade e relacionamentos essenciais
customer	UUID; documento único; active controla elegibilidade no login CPF
vehicle	UUID; placa única; associação a customer (nullable no mapeamento atual)
worker	UUID; perfil, email único, password_hash e active
work_order	UUID; cliente e veículo obrigatórios; funcionário opcional; status e datas do ciclo
work_order_history	UUID; OS obrigatória; status anterior/novo e instante da transição
estimate	UUID; OS obrigatória; status e valores de peças/mão de obra/total
estimate_item	UUID; orçamento e peça obrigatórios; quantidade, preço e total
parts	UUID; quantidade de estoque, mínimo e @Version para concorrência otimista
service_item	UUID; catálogo, preço base, duração estimada e active
work_order_service	UUID; OS obrigatória; catálogo opcional; descrição/preço registrados
estimate_decision_token	UUID atribuído pelo domínio; IDs de OS/orçamento, decisão e validade/consumo
app_users	Identificador numérico; username único, credencial, role e active; entidade de autenticação separada

As relações JPA @ManyToOne são relações mapeadas, sujeitas ao DDL efetivamente aplicado. `estimate_decision_token.work_order_id/estimate_id` são campos UUID, não associações JPA: o diagrama usa linhas tracejadas para vínculos lógicos e não afirma existência de FKs físicas. `app_users` possui entidade JPA no código atual; a documentação legada que o chamava de tabela sem entidade precisa ser substituída por este retrato atualizado. Valores de credenciais e dados pessoais não integram o diagrama.

Cardinalidades e consistência

- Cliente pode ter vários veículos/OS; OS pertence a um cliente e veículo.
- Funcionário pode atender várias OS, e sua atribuição é opcional.
- OS possui histórico, serviços e orçamentos. A modelagem não impõe 1:1 entre OS e orçamento.
- Orçamento contém itens que referenciam peças. Valores monetários usam BigDecimal/decimal.
- Serviço realizado pode manter a referência do catálogo ou apenas o retrato de descrição/preço.

- O estoque usa versão otimista. A consistência do fluxo também depende dos casos de uso e transações.

- Há constraints para estados canônicos em work_order/work_order_history; estados históricos

incompatíveis são tratados pela migration V2, preservando campos legados.

Evolução do schema e pontos de revisão

V1 cria o schema; V2 ajusta o ciclo da OS e seu histórico; V3 adiciona active em customer. Flyway migrate-at-start está habilitado, mas Hibernate schema-management.strategy=update continua configurado fora dos testes. Logo, Flyway ainda não é a única autoridade do DDL. A revisão não executou DDL nem consultou dados no banco.

A melhoria recomendada é uma baseline completa e migrations explícitas para criação/evolução, seguida de validate no runtime. Isso exige planejar migração do banco já existente; não foi aplicado automaticamente. Não se afirma que PK/FK/índices do schema vivo foram inspecionados.

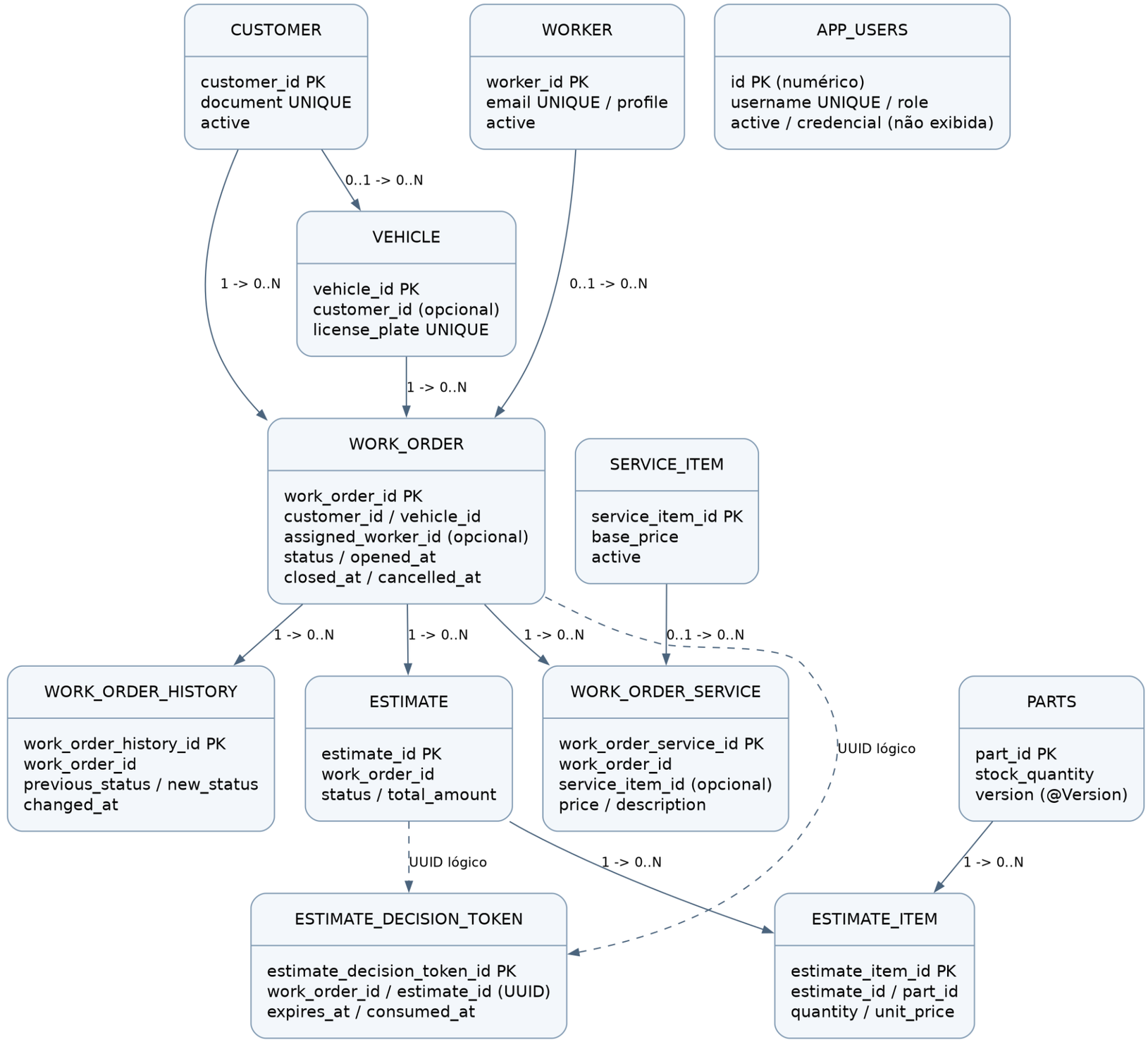
Performance e indicadores de negócio

Documento único/placa/username/email de funcionário têm unicidade no mapeamento. Consultas por cliente, fila operacional, datas e histórico precisam de índices avaliados por planos reais antes de otimizar. Esta documentação não inventa índices nem resultados EXPLAIN.

O endpoint atual de tempo médio calcula abertura->fechamento das OS concluídas. Isso não entrega volume por dia nem médias separadas de Diagnóstico/Execução/Finalização no Grafana. Essas agregações devem usar dados persistidos e históricos de transição, definir timezone, tratamento de cancelamentos/reentrada e semântica de Finalização. Contadores apenas na memória de um pod não substituem agregação correta com réplicas/reinícios.

Fonte: entidades de persistência da aplicação e migrations.

Modelo relacional



Fonte editável e versão vetorial no diretório docs/fase3/diagramas. Diagrama de homologação; limites e caminhos alternativos no texto.

RFC-001 - AWS e serviços gerenciados

Status: justificativa técnica da implementação existente, aberta à revisão da equipe.

Problema

O desafio exige Gateway, função serverless, Kubernetes escalável, banco gerenciado e IaC. O Learner Lab limita IAM e oferece credenciais temporárias. A solução precisa ser demonstrável sem esconder essas restrições nem criar dependência de acesso administrativo não disponível.

Solução adotada

AWS API Gateway HTTP, Lambda Go, EKS, ECR, RDS PostgreSQL, S3 para state e CloudWatch. Terraform provisiona recursos AWS; Helm instala observabilidade depois que o cluster está acessível. Quatro repositórios mantêm fronteiras de entrega e workflows por ambiente.

Alternativas e critérios

Alternativa	Avaliação
EKS + RDS + Lambda	Alinha os requisitos e aproveita o Lab; exige gerir compatibilidade Kubernetes e limites IAM
Kubernetes autogerido + banco em VM	Aumenta operação e banco em VM não atende diretamente o requisito de banco gerenciado
Gateway/autenticação no pod	Possível tecnologicamente, mas mantém acoplamento de disponibilidade e não substitui a função serverless exigida
Outra nuvem	Viável pelo enunciado; exigiria reprovisionamento e credenciais fora do ambiente disponibilizado

Consequências

Roles preexistentes do Lab restringem privilégio fino. Homolog e produção compartilham EKS; RDS é por ambiente. O ambiente não promete tolerância a falha de região/AZ. Custos dependem principalmente de EKS, nós, RDS e dois NLBs internos; não foi efetuado cálculo de custo mensal.

Critérios de aceitação: apply revisado, workloads prontos, tráfego via Gateway, testes de negação de acesso e evidência de monitoramento. Evolução comercial exige IAM dedicado, isolamento e HA dimensionados por requisitos medidos.

RFC-002 - PostgreSQL gerenciado e evolução de dados

Status: justificativa técnica da implementação existente, aberta à revisão da equipe.

Decisão e critérios

Adotar PostgreSQL 16 em Amazon RDS. As operações do domínio requerem transações, relacionamentos e agregações; a stack já utiliza Hibernate Reactive, JDBC/Flyway e pgx. RDS atende ao requisito explícito de banco gerenciado. Ver modelo e justificativa.

Alternativas

MySQL gerenciado é funcionalmente possível, porém acrescenta migração de dialeto/driver e não há evidência de ganho neste sistema. NoSQL não elimina a necessidade de consistência entre OS, orçamento e estoque, e exigiria redesenho do modelo. Banco em container não atende à intenção de serviço gerenciado nem transfere operação/backups à nuvem.

Integridade e operação

Preservar UUIDs do domínio, decimal para dinheiro, unicidade onde mapeada, estados canônicos e transações do caso de uso. O banco é privado, com transporte TLS. Backups seguem Terraform de cada ambiente; Multi-AZ é desativado no Lab. Schema e credenciais não devem ser colocados em artefatos públicos.

Pontos em aberto

DDL ainda combina Flyway e schema-management=update. A baseline completa e a validação exclusiva por migrations são uma evolução planejada. Índices devem ser justificados por consulta/plano medido; esta revisão não fez acesso SQL. Definir métricas diárias e duração por etapa a partir do histórico antes de afirmar conformidade integral de observabilidade.

RFC-003 - CPF, JWT RS256 e autorização

Status: justificativa técnica da implementação existente, aberta à revisão da equipe.

Solução

A Lambda customer-auth valida o CPF e o status do cliente no RDS antes de emitir JWT. A Lambda jwt-authorizer valida assinatura/issuer/expiração para as rotas protegidas do HTTP API. A aplicação aplica RBAC e propriedade dos recursos. As funções são distintas: emitir uma identidade e aceitar uma identidade são responsabilidades diferentes.

Alternativas consideradas

JWT apenas na API simplifica o caminho interno, mas não demonstra o Authorizer no Gateway. Tokens opacos com sessão central permitem revogação mais imediata, mas adicionam consulta de sessão ao caminho de autenticação. Um provedor de identidade gerenciado pode melhorar segurança de uso comercial; não substitui automaticamente a função CPF exigida no desafio.

Consequências e limites

RS256 permite validar com chave pública sem distribuir a privada ao Authorizer. As chaves privadas continuam necessárias aos emissores e devem permanecer em secrets. Cache zero no Authorizer simplifica os testes de autorização e evita cache de uma decisão anterior. A emissão via CPF sem fator adicional é uma escolha acadêmica, insuficiente como única prova forte de identidade em um produto comercial.

Cliente inativo não recebe novo token. Um token já emitido pode continuar válido até expirar; não se afirma revogação imediata. CPF/JWT/senhas não devem ser exibidos no vídeo/logs.

Verificação

Matriz real em homolog: login conhecido e acesso CUSTOMER 200; documento sem cadastro e ausência de token 401; Bearer malformado, token alterado e rota operacional CUSTOMER 403. O status HTTP de negócio deve ser distinguido de falha 5xx de integração.

ADR-001 - Comunicação síncrona e fronteiras de autorização

Status: estado implementado documentado nesta revisão.

Contexto

O usuário precisa obter respostas da API e executar operações transacionais. Gateway, autenticação, RBAC e regras de negócio têm responsabilidades separadas.

Decisão registrada

Manter HTTP síncrono: HTTPS no Gateway, integrações Lambda para autenticação/Authorizer e VPC Link/NLB interno para a API. O código da aplicação usa portas/adaptadores e fluxo reativo. A autorização criptográfica do Gateway precede RBAC e propriedade de recurso na API. Management é um Service separado, sem rota pública. Grafana usa sua própria autenticação.

Consequências

O cliente observa imediatamente o status da operação. Falhas de rede/backend precisam de correlação e monitoramento. Não há contrato de fila/outbox/exatamente-uma-vez. SMTP é uma integração do adaptador da API e sua falha não deve ser confundida com erro de autenticação. Tokens, CPF e senhas ficam fora dos logs/documentos. Endpoints públicos são exceções explícitas.

Evidência

api-gateway.tf, scripts de smoke, controllers @RolesAllowed e portas/adaptadores da aplicação.

ADR-002 - HPA de CPU e memória

Status: estado implementado documentado nesta revisão.

Contexto

A API precisa aumentar capacidade em picos sem depender de intervenção por requisição. O Lab tem capacidade finita e a stack de observabilidade compartilha os nós.

Decisão registrada

Usar autoscaling/v2 para oficina-mecanica: mínimo 1 e máximo 6 réplicas; alvo CPU 60% e memória 80%. Metrics Server fornece recursos. Scale-up permite 2 pods a cada 15 segundos, sem janela inicial; scale-down estabiliza por 60 segundos e remove 1 pod por 30 segundos. Deployment usa rollout com maxUnavailable=0 e maxSurge=1, além de probes de saúde.

Consequências

HPA escala pods, não nós. O node group é gerido por Terraform; este ADR não afirma Cluster Autoscaler. Uma réplica mínima é economia acadêmica, sem garantir alta disponibilidade. O teto de 6 é um limite, não comprovação de que todas as réplicas cabem após a instalação de Prometheus/Loki/Tempo. Medir requests, limites, scheduling e carga antes da gravação.

Verificação e reversão

Conferir kubectl get hpa/top, gerar carga sintética limitada, observar aumento e redução, registrar readiness e ausência de erros. Reverter thresholds/políticas por PR e rollout controlado se houver oscilação. Não usar dados reais nem tráfego irrestrito para demonstração.

Fonte: hpa.yaml.

ADR-003 - Observabilidade compartilhada no Learner Lab

Status: estado implementado documentado nesta revisão.

Contexto

O requisito permite escolha livre de ferramenta. O cluster precisa expor sinais técnicos, logs e traces sem abrir publicamente o management. O EBS CSI ficou CREATE_FAILED porque não conseguia obter credenciais; simulação IAM negou alterar trust/permissões da LabRole.

Decisão registrada

Prometheus/Grafana para métricas; Alloy/Loki para JSON; Alloy/Tempo para tracing; Blackbox para a sondagem pública; logs Lambda/Gateway no CloudWatch. A stack vive em observability. Somente Grafana é roteado publicamente pelo Gateway/NLB interno com login obrigatório. Dois dashboards versionados na pasta FIAP substituem os genéricos; a visão geral é a home.

No Lab, usar emptyDir e retirar o add-on EBS defeituoso após plano controlado que confirmou PVCs Pending sem volume. Dashboards/configurações são reprovisionados do Git; histórico e alterações manuais no Grafana podem ser perdidos quando os pods forem recriados.

Consequências e limites

A decisão viabiliza a demonstração, sem garantia de retenção durável. Não se declara que uma configuração de retenção em horas/dias garante sobrevivência a recriação de pod. Produção comercial deve reavaliar armazenamento e IAM dedicado. A Lambda tem tracing opcional; o caminho privado/exportação não foi demonstrado nesta revisão.

O deploy reconcilia ServiceMonitor da aplicação, pois o primeiro deploy pode ocorrer antes do CRD, e aplica a Probe após o Blackbox. Dados de negócio exigem instrumentação/agregação própria; gráficos de HTTP/JVM não equivalem a volume diário e tempo médio de cada etapa da OS.

Verificação

Workflow de publicação, login/banco Grafana HTTP 200, dois dashboards, séries API/uptime, logs JSON e traces reais. Testar alertas específicos de OS e a entrega/notificação antes de declarar encerrada a conformidade de monitoramento.

Matriz de atendimento - Fase 3

Revisão: 30/10/2026 (BRT). Base: enunciado fornecido pelo usuário, páginas 2-5, branches homolog e evidência de execução. O usuário confirmou que ainda não produziu vídeo nem PDF final de entrega. Os documentos e diagramas deste pacote foram preparados nesta revisão. **O projeto ainda não está integralmente pronto para a entrega.**

Estados: **Comprovado** = evidência observada; **Implementado** = fonte/configuração existente, sem demonstração completa; **Parcial** = cobertura incompleta; **Pendente** = item ainda faltante.

Infraestrutura, segurança e CI/CD

Requisito	Estado	Evidência e limite
API Gateway de entrada	Comprovado	API pública em HTTPS, integrações privadas e Swagger/OpenAPI acessíveis
Função serverless autentica CPF	Comprovado	Cliente sintético ativo confirmado; login 200; fonte consulta status no RDS
JWT e proteção de rotas sensíveis	Comprovado	CUSTOMER 200; sem token 401; Bearer inválido/alterado e RBAC 403
CPF inválido/inativo	Implementado	Validação e checagem active no código/testes; não foi desativado cliente para teste de runtime nesta revisão
Quatro repositórios separados	Comprovado	app, lambda-auth, infra-k8s e infra-db existentes e públicos
CI/CD funcional dos quatro	Comprovado em homolog	Runs de entrega concluídos dos quatro; o DB não foi reaplicado com a sessão atual
main exige PR	Comprovado	Regras ativas de rulesets nos quatro; a API antiga de branch protection sozinha não refletia essas regras
Deploy automático homolog/production	Implementado	Triggers por branch nos quatro workflows; nenhuma execução em production encontrada
Banco gerenciado	Comprovado	RDS PostgreSQL provisionado e consumido por aplicação/autenticação; sem inspeção SQL nesta revisão
Kubernetes e escalabilidade	Implementado	EKS, Metrics Server e HPA 1-6 réplicas; teste atual de carga/subida/descida ainda deve ser gravado
Provisionamento Terraform	Comprovado	Applies homolog e reconciliação com plano/lock; stack Helm é etapa separada
Dockerfile quando aplicável	Implementado	Aplicação tem Dockerfile; Lambdas são binários ZIP; repositórios Terraform não necessitam container de runtime
Avaliador soat-architecture	Comprovado	Acesso confirmado nos quatro repositórios via API do GitHub

Observabilidade

Requisito	Estado	Evidência e limite
Integração com ferramenta de observabilidade	Comprovado	Stack Prometheus/Grafana/Loki/Tempo/Alloy/Blackbox publicada
Grafana simples e acessível	Comprovado	31 dashboards reduzidos a 2 na pasta FIAP; home aponta para a visão geral; login e banco HTTP 200
CPU e memória Kubernetes	Comprovado	Exporters/targets disponíveis e dashboard de recursos versionado
Coleta da API	Comprovado	2 targets oficina-mecanica-management em homolog, ambos UP após reaplicar ServiceMonitor
Latência e tráfego HTTP	Parcial	Painéis e filtro de histogramas existem; auditoria real não encontrou http_server_requests_seconds_count/bucket. Existem métricas de conexões/bytes, que não substituem latência p95
Healthchecks/uptime	Comprovado com limite	Probe pública com sucesso=1, readiness/liveness nos manifests. Probe mede Swagger; não representa todas as operações/integrações
Alertas de falha no processamento de OS	Pendente	Existem alertas técnicos genéricos; faltam sinal/regra específica de OS, demonstração firing/resolved e evidência do destino de notificação
Logs JSON	Comprovado	Amostra Loki contém somente entradas JSON; nenhum conteúdo foi exportado para este documento
Correlação entre requisições	Parcial	Filtro X-Request-ID/MDC existe; amostra teve traceId, mas não requestId. Falta evidenciar um fluxo correlacionado real
Traces em execução	Pendente de evidência	Tempo consultado sem traces; presença de traceId no log não comprova exportação/consulta do trace. Lambda OTLP é opcional e não foi comprovado
Volume diário de OS	Pendente	Nenhuma série/agregação de negócio correspondente nos dashboards/auditoria
Tempo médio por etapa	Pendente	Endpoint atual mede abertura->fechamento, sem separar Diagnóstico/Execução/Finalização no Grafana
Erros/falhas de integrações	Parcial	Há sinal HTTP 5xx e logs, sem painel/métrica dedicada das integrações como SMTP/DB/autenticação

Documentação e entrega

Requisito	Estado	Evidência e limite
Diagrama de componentes completo	Documentado	ARQUITETURA.md + componentes PNG/SVG/Mermaid
Sequência de CPF e abertura de OS	Documentado	SEQUENCIAS.md + dois diagramas exportáveis/editáveis
RFCs	Documentado	RFCs cloud, PostgreSQL e autenticação, com justificativas e limites
ADRs	Documentado	Comunicação, HPA e observabilidade; complementam os ADRs existentes da aplicação
Justificativa do banco / ER / relacionamentos	Documentado	MODELO-DE-DADOS.md + ER atualizado, inclusive active, app_users e tokens de decisão
README de cada repo com pro pósito/stack/deploy/diagrama	Parcial	Existem os quatro; faltam alinhamento/link explícito de Swagger/deploy/documentação em alguns e atualização da referência ao vídeo antigo
Vídeo atual <=15 min	Pendente do usuário	Roteiro preparado; usuário gravará após concluir as lacunas técnicas
PDF único para o Portal	Pendente do vídeo	Folha de entrega preparada; dossiê técnico não é submissão final enquanto faltar vídeo e conformidade

Evidências reproduzíveis

- Publicação do Grafana simplificado e reconciliação.
- Auditoria: dois dashboards, home, API targets, métricas, uptime, logs e Tempo.
- Matriz HTTP real do Authorizer.
- Confirmação da conta sintética ativa.
- Deploy da aplicação em homolog.
- Deploy da Lambda em homolog.
- Apply de banco em homolog.

Ordem para fechar a fase

1. Restabelecer as séries HTTP/histogramas e demonstrar trace real no Tempo, correlacionado ao log.

2. Implementar as agregações persistentes de volume diário e duração por etapa; definir a semântica de Finalização/cancelamentos.

3. Instrumentar falhas de OS/integrações e demonstrar alerta disparando e resolvendo.

4. Atualizar links/READMEs dos quatro repositórios e registrar evidência de HPA/pipelines. Conferir credenciais do DB respeitando a restrição atual de secrets a três repositórios; não ampliar esse escopo automaticamente.

5. Usuário grava o vídeo; conferir acesso/duração, preencher folha e gerar o PDF final de submissão.

A política do Lab impediu o uso de EBS CSI com a identidade necessária. emptyDir permitiu a homologação, mas histórico de telemetria é volátil. O enunciado não determina EBS, Multi-AZ ou um fornecedor específico de monitoramento: essas limitações não são transformadas aqui em exigências inventadas, mas devem ficar claras na apresentação.

Atualização operacional - 31/10/2026 (BRT)

- Deploy homolog concluído: https://github.com/15SOAT-JVGG/oficina-app/actions/runs/41158248740
- Seed espaçado aprovado: seis OS, uma por estado; HTTP 200/201, sem 503 nesta execução: https://github.com/15SOAT-JVGG/oficina-app/actions/runs/41158958643
- Carga separada: 211 tentativas, oito 503; teste reprovado corretamente: https://github.com/15SOAT-JVGG/oficina-app/actions/runs/41158493007
- Réplicas Ready passaram de duas para quatro durante a carga e retornaram a duas nas amostras do seed posterior. Isso comprova reação do HPA, sem comprovar estabilidade.
- NLB cross-zone habilitado, mas 503 persistiu. Há também falhas SMTP por limite de envio; elas não explicam os 503 da carga de login, que não envia e-mail. Causa ainda aberta.
- Indicadores de negócio, histogramas HTTP, traces e alertas específicos continuam pendentes.

Roteiro simples – demonstração da Fase 3

Meta: 12 a 14 minutos. Limite do enunciado: 15 minutos. Gravar a versão final após fechar as pendências da matriz; não apresentar painel vazio como indicador atendido.

Preparação

- Abrir o diagrama de componentes, os quatro repositórios e uma execução de deploy concluída.

- Abrir Swagger, cliente HTTP e os dois dashboards do Grafana.

- Usar apenas dados sintéticos. Ocultar CPF, senha e JWT no enquadramento.

- Fazer o seed antes da gravação: seed_only, 6 OS. A carga é separada: load_only,

180 segundos, concorrência 2. Esses valores menores ainda precisam de validação de carga.

- Não executar carga junto com rollout. Não repetir seed para gerar pressão: cada lote permanece.

1. Apresentação e arquitetura - 1 minuto

Fala sugerida: “Esta é a aplicação da oficina. Organizamos a solução em quatro repositórios: aplicação, autenticação serverless, infraestrutura Kubernetes e banco gerenciado.” Mostrar o diagrama e apontar Gateway, Lambda, EKS, PostgreSQL e observabilidade.

2. Pipeline e deploy - 2 minutos

Mostrar um PR, proteção da main, checks e uma execução concluída em homologação. Apontar teste, publicação da imagem, rollout e smoke pelo Gateway. Mostrar também os workflows dos outros três repositórios. Dizer que produção tem triggers configurados, mas sua execução ainda não está comprovada nesta homologação.

3. Autenticação e API protegida - 2 minutos

Fazer login com CPF sintético ativo sem revelar seu valor nem o JWT. Mostrar apenas HTTP 200. Consumir uma rota CUSTOMER com sucesso. Repetir sem token (401) e tentar uma rota operacional como CUSTOMER (403). Explicar que a Lambda verifica cadastro/status e emite JWT.

4. Ordem de serviço - 2 minutos

Como funcionário, abrir uma OS e mostrar sua consulta. Usar o lote preparado para mostrar Recebida, Diagnóstico, Aguardando aprovação, Em execução, Concluída e Entregue. Explicar o orçamento e a reserva de estoque, sem expor dados pessoais.

5. Monitoramento e escala - 3 minutos

No Grafana, abrir Visão geral e Infraestrutura. Mostrar uptime, CPU, memória e réplicas. Iniciar uma carga leve separada do seed, observar o HPA e sua recuperação. Não prometer atingir seis pods. Se ocorrer 503, interromper e registrar a falha; não esconder o resultado. Mostrar latência/erros, volume diário de OS, tempo médio por etapa e falhas de integrações somente após implementar e validar esses indicadores, ainda pendentes na revisão atual.

6. Logs, traces e alertas - 2 minutos

Selecionar uma requisição de teste, mostrar log JSON e o trace correspondente no Tempo. Demonstrar alerta de falha de OS disparando e resolvendo. Esses exemplos precisam estar validados antes da gravação final: logs com traceId, sozinhos, não comprovam trace exportado.

7. Documentação e encerramento - 1 minuto

Mostrar componentes, as duas sequências, ER, RFCs e ADRs nos arquivos separados. Encerrar com links dos quatro repositórios e confirmação de acesso do avaliador.

Depois da gravação

Publicar no YouTube/Vimeo como público ou não listado, conferir acesso e duração de até 15 minutos e incluir o link na folha de entrega. O Portal pede um PDF único com links dos repositórios, vídeo, documentação e confirmação do avaliador. Os arquivos separados são material complementar; não substituem o vídeo nem o PDF único.

Folha de entrega - preencher após o vídeo

Estado: preparação. Ainda não é uma submissão final. O usuário confirmou que gravará o vídeo depois de concluir os requisitos técnicos. Não há link de vídeo atual da Fase 3.

Repositórios

1. https://github.com/15SOAT-JVGG/oficina-lambda-auth
2. https://github.com/15SOAT-JVGG/oficina-infra-k8s
3. https://github.com/15SOAT-JVGG/oficina-infra-db
4. https://github.com/15SOAT-JVGG/oficina-app

Documentação

- Índice: https://github.com/15SOAT-JVGG/oficina-infra-k8s/tree/homolog/docs/fase3
- Componentes, sequências, ER, RFCs, ADRs e validação: arquivos vinculados no índice.
- Swagger: https://dcs5hfax0e.execute-api.us-east-1.amazonaws.com/q/swagger-ui/
- OpenAPI: https://dcs5hfax0e.execute-api.us-east-1.amazonaws.com/q/openapi?format=json
- Grafana: https://dcs5hfax0e.execute-api.us-east-1.amazonaws.com/observability/

Os links de homologação dependem da sessão/recursos do Learner Lab e devem ser conferidos antes da entrega. Login do Grafana usa admin e secret protegido; a senha não integra o PDF.

Acesso do avaliador

A consulta ao GitHub confirmou `soat-architecture` com acesso aos quatro repositórios em 30/10/2026 (BRT). Reconfirmar esse acesso antes de enviar o PDF no Portal do Aluno.

Pendências finais

- Vídeo atual (YouTube/Vimeo, público ou não listado, até 15 minutos): **pendente de gravação pelo usuário**.
- Fechar pendências técnicas registradas na matriz de validação.
- Completar links dos READMEs e promover/documentar a revisão nos quatro repositórios.
- Gerar PDF único com quatro links, vídeo, documentação e confirmação do avaliador.

O dossiê arquitetural desta revisão pode apoiar a apresentação, mas não substitui o PDF final enquanto o vídeo e as pendências de conformidade estiverem em aberto.